Context-Aware Campus Assistant

Nicola Tinari
Dept. of Computer Science and Engineering
AlmaMater Studiorum - University of Bologna
Bologna, Italy
nicola.tinari@studio.unibo.it

Abstract - This document presents a platform, called the Context-Aware Campus Assistant, designed to provide support to students on various campuses on the places and services that could be useful to them during their academic time in a city that is often not even known. The application does not stop only at the visual support on the map, but also tries to suggest to the student which places (or rather POIs) in the vicinity could arouse his interest through a system of notifications and geofencing based on a precise ranking rule that takes into consideration: the student's preferences, a time factor and distance.

Keywords*: Context-Aware Systems, Location-Based Services (LBS), Android MVVM, FastAPI, PostGIS, Docker Containerization, Multi-Criteria Decision Making.*

I. INTRODUCTION

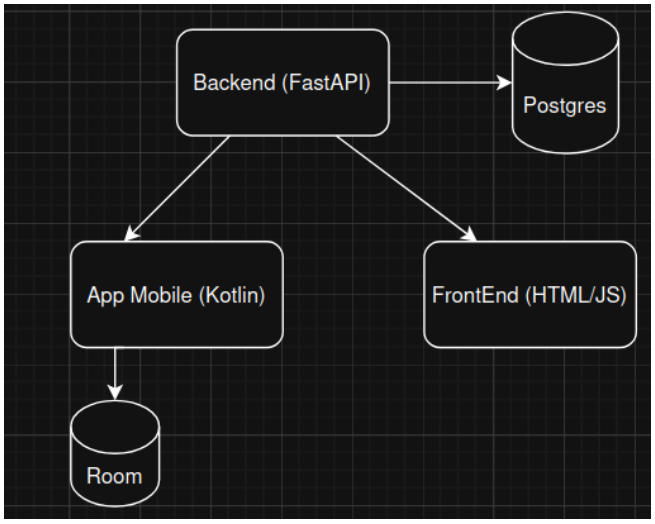
The life of a university student takes place in a complex ecosystem characterized by a continuous interaction between physical mobility, the need for information and requests for available services. Students, especially those enrolled on a campus located in a city other than their place of residence (called "off-site"), need effective tools to find their way around the various services available such as: study rooms, libraries, canteens and public transport stops. Most existing mobility support solutions are inherently static, i.e. they provide maps or lists of places without taking into account the dynamic context in which the learner is immersed. Context refers to any information that can be used to characterize the situation of an entity relevant to the interaction between a user and an application. In a university environment, the value of information is closely linked to this context: knowing the location of a library is irrelevant if it is closed or if the distance is incompatible with the user's schedules and mobility constraints. The Campus Assistant platform is designed to overcome this limitation by transforming the mobile device from a passive consultation tool into a proactive assistant.
The system combines three contextual dimensions: the user's spatial location, the time context (opening hours and personal agenda) and the user profile (favorite categories and means of transport), so that the same location generates different suggestions depending on the time of day and the individual user.
The main features of the project are:

- A contextualized recommendation engine that classifies nearby points of interest (POIs) via an explicit scoring rule based on distance, times, and user preferences
- A Geofencing support with contextual entry/exit notifications, along with a personal history of suggestions received and an explicit user feedback mechanism (useful/not helpful)

- A personal agenda integrated with a proactive notification system that alerts the user before upcoming commitments (15 minutes before)
- An offline mode based on local data persistence, which guarantees access to POIs and the agenda (even if read-only) even in the absence of a network
- An administrative web dashboard for CRUD management of POIs and spatial analysis (usage statistics and heat maps)
- Full containerization of the platform using Docker Compose, which allows deployment with a single command.

II. PROJECT'S ARCHITECTURE

The Context-Aware Campus Assistant adopts a three-tier client-server architecture, shown in Figure 1. There are two independent clients, the native Android application and the administrative web dashboard, which communicate with a RESTful backend via HTTP calls. The backend in turn interacts with the database (Postgres) in which all entities and geometries are persistently stored. All server-side components (backend, database, and web server) are also containerized and orchestrated via Docker Compose.



(Fig. 1 Three-tier architecture of the platform)

II.A App Mobile

The Android client follows the MVVM (Model-View-ViewModel) architectural pattern extended with the Repository pattern (the only object that makes the API calls). This tiered structure imposes a clear separation of responsibilities, improving the testability and maintainability of the application. Each level has its own distinct responsibility:

- The View is implemented with the Jetpack Compose toolkit [2]. Its sole responsibility is to render the user interface according to the detected state. This architectural layer contains no business logic and simply captures user interactions and delegates management to the functions exposed by the corresponding ViewModel.
- The ViewModel layer contains the state of the UI and coordinates the logic of the application. It processes the events received from the View, queries the appropriate

Repositories, and publishes status updates through the mutableStateOf Compose observable state handlers, which automatically trigger the recomposition of the observing View. Asynchronous operations, such as continuously collecting GPS updates or retrieving remote data, are handled through Kotlin Coroutine, avoiding any blocking of the UI thread.

- The Model defines the application domain. It includes the Entities stored locally through the Room library, their Data Access Objects (DAOs), which form the foundation of offline support, and the Data Transfer Objects (DTOs) used for serialization of network payloads and the domain models used by ViewModels. DTOs maintain structural consistency with entities defined in the backend
- The repository mediates all access to the data: the repositories are the only components allowed to reach the local database or remote APIs. Each backend entity is managed by a dedicated Repository that acts as a single source of truth for that domain and implements a "remote-first" strategy with automatic fallback: data is requested first from remote REST endpoints via the Retrofit library, and in the event of a network failure (i.e., when an IOException exception is thrown), the request is handled transparently by the local Room cache.

II.B Backend

The backend consists of a RESTful server developed in Python based on the FastAPI framework [1]. This choice is motivated by three factors: the high performance in the management of requests of its execution model, the native integration with Pydantic that provides the automatic validation of all incoming and outgoing payloads and the automatic generation of interactive API documentation compliant with the OpenAPI specification (Swagger UI), systematically used to document and test the exposed endpoints. Internally, the backend applies the Service pattern combined with Dependency Injection (DI). Routing modules act solely as controllers: they expose HTTP endpoints, handle requests and responses, and delegate all business logic to dedicated service classes (e.g., POIService, AgendaUtenteService), each associated with a single logical domain (DB table) according to the principle of single responsibility. The interaction between controllers, services, and databases is orchestrated through the Depends() structure of the FastAPI. Specifically, database sessions are generated by a factory function (get_session()) and injected into services at runtime, thus avoiding repeated code. For each logical entity, the models are segmented by role, following the model recommended by the framework:

- Base: contains the domain attributes shared by all variants, along with the declarative validation constraints (e.g., maximum field length, non-nullable fields) imposed by Pydantic

- Entity (the table model): extends the Base variant with the primary key and is mapped to the corresponding database table;
- Create/Update: Define input payloads for insertion and editing; all fields of the Update variant are optional, in order to support partial updates;
- Public: Defines the output payload, ensuring that the APIs return only the data that is strictly necessary and avoid database metadata.

Where appropriate, additional derived models extend this hierarchy for specific use cases (e.g., a POI variant enriched with user-calculated distance, returned by proximity queries).

II.C Data Management

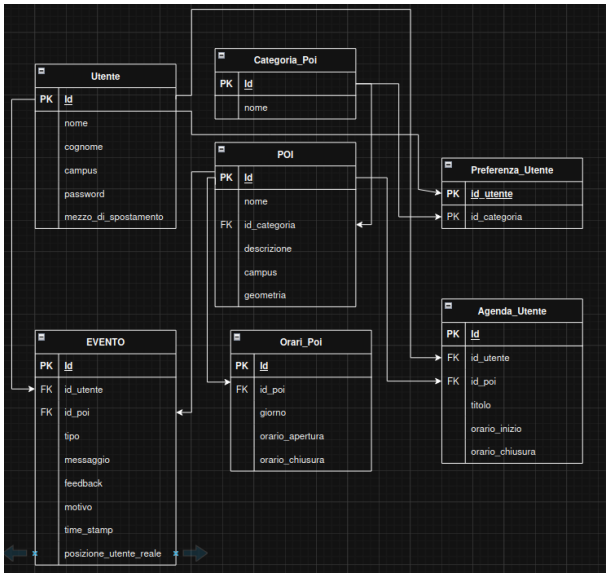
The central persistence layer is based on PostgreSQL extended with the PostGIS module, which allows for the native storage of geographical features and their spatial indexing. Geometries are stored using the generic GEOMETRY type with SRID 4326, ensuring direct compatibility with GPS coordinates produced by the mobile client. Since EPSG:4326 expresses coordinates in angular degrees, all metric calculations are performed first by re-projecting the geometries into EPSG:32632 via ST_Transform and then applying the desired function (ST_Distance or ST_Within) before converting back to EPSG:4326. Delegating these operations to the database avoids transferring the entire dataset of POIs to the application layer and exploits the spatial index, keeping the response time of the pipeline of contextualized suggestions low. On the serialization side, geometries are converted into GeoJSON objects at the API boundary, providing clients with a standard and easily parsable geographic format. For demonstration purposes, the database was populated with real points of interest from the campus area of Bologna, Forlì and Cesena. Data retrieved via QGIS and imported into PostGIS via a GeoPandas-based pipeline (after a slight manual review to ensure data consistency).

II.D Frontend

The supervision layer is a lightweight single-page web client, served by the Nginx container. It is implemented in HTML5, CSS, and simple JavaScript: all interactions with the platform take place via asynchronous calls (Fetch API) to the backend REST endpoints. The interface integrates Leaflet.js, based on OpenStreetMap tiles, for interactive visualization of points of interest (POIs), the leaflet.heat plugin for rendering the spatial heatmap of contextual events, and Chart.js for aggregated usage statistics. Architecturally, the dashboard is a client of the same REST APIs used by the mobile application: specifically, filtering POIs by category, campus, and time range is performed by querying the same filtering endpoint used by the app, which ensures data consistency across clients and avoids any duplication of business logic on the presentation side. The dashboard extends this shared API surface only with the administrative operations for which it is responsible, namely the CRUD management of the POI catalog and the use of analytics endpoints.

III. PROJECT'S IMPLEMENTATION

III.A Data management



(Fig. 2 Relational schema of the PostgreSQL/PostGIS database)

Figure 2 shows the relational schema of the system database, built on PostgreSQL/PostGIS. The schema revolves around the Point Of Interest (POI) entity. Each Point of Interest belongs to one of the categories stored in the CATEGORIA_POI table ("biblioteca", "sala_studio", "mensa", "ufficio", "segreteria", "fermata", "noleggio_bici", "stazione", "benzinaio"), has a campus attribute used for geographical filtering and is described by a PostGIS geometry stored in the EPSG:4326 reference system, natively compatible with the GPS coordinates produced by the mobile client. The data model then intersects the spatial dimension with the user's personal and temporal dimensions. The UTENTE table stores the user profile, including the mezzo_di_spostamento ('A PIEDI', 'BICI A NOLEGGIO', 'AUTOBUS', 'TRENO', 'MOTO', 'AUTO', 'ALTRO') and the password that allows the user to access their personal mobile app profile. PREFERENZA_UTENTE is an associative table that implements the many-to-many relationship between users and their favorite categories and is exploited for the generation of suggestions (section III.B). The time dimension of the services is represented by the ORARIO_POI table, which stores an opening interval for each day of the week (coded as an integer, from 0 = Sunday to 6 = Saturday) for each POI, by default the absence of time records for a POI is interpreted as a service that is always active 24 hours a day. This table helps the system determine whether the service is currently available at the time of the request. Each user also has a personal agenda (AGENDA_UTENTE), which associates the activities with a title, a time interval and the POI in which they take place. In addition to acting as an integrated calendar, the agenda activates a proactive notification mechanism that alerts the user when a recorded activity is about to start within 15 minutes (see section III.C). Finally, the tracking of contextual interactions is managed by the EVENTO table, which records every contextualized decision taken by the system: suggestions generated, agenda alerts, POI selections and entry/exit triggers from geofencing produced by the client (event types: Avviso_agenda, suggerimento, poi_selezionato, geofencing_enter, geofencing_exit). Each event records the timestamp, the user and POI involved, the user's exact location (posizione_utente_reale), and, crucially, a human-readable reason (reason) field that

explains why the event was generated (e.g., highest-scoring POIs for the current context, upcoming agenda engagement, geofencing triggers). Each event also contains a user-editable feedback flag (Useful/Not Useful). However, only feedback associated with contextual suggestions is taken into account in the calculation of statistics, and by default a dismissed suggestion is considered as "Not Useful".

III.B Backend

The backend implementation follows the tiered organization introduced earlier, with routing modules, service classes, and segmented data models. The APIs are modularized into dedicated routers (one per logical domain), and each path contains a brief description and declares its typed input and output patterns. In this way, the documentation automatically generated by the framework is complete and immediately ready for use in the first tests:

1)REST API
Table I summarizes the endpoints used by the two clients, grouped by router with their input parameters. Response formats follow the convention described below patterns. The entire set of APIs can be accessed through the interactive OpenAPI documentation.

Regarding response formats, unless otherwise specified each endpoint returns the Public variant of its domain model (e.g., UtentePublic, POIPublic, EventoPublic) or a list thereof, according to the model segmentation described in Section II.B. The exceptions are: /poi/nearby-with-distance returns List[POIDistance], /recommendations/top_20_ranking returns List[RankingResult], /agenda/imminenti returns List[AgendaUtenteContext], /geofence/config/{id_utente} returns List[GeofenceConfigResponse], /geofence/trigger and /recommendations/startup-suggestion return the newly created EventoPublic, /analytics/dashboard returns DashboardResponse and /analytics/heatmap/pois returns List[HeatmapPoint].

TABLE I — REST API ENDPOINTS

Endpoint / Path	**Parameters / Input**
POST /auth/register	UtenteCreate (body)
POST /auth/login	LoginRequest (body)
PATCH /utenti/{id}	UtenteUpdate (body)
GET /preferenze/utente/{id_utente}	path param
POST /DELETE /preferenze/	PreferenzaUtenteCreate (body) / id_utente, id_categoria (query)
GET /poi/	–

GET /poi/nearby-with-distance	lat, lon, radius=2000 (query)
GET /poi/filter	lat, lon, id_categoria[], max_distance_meters, orario_apertura, orario_chiusura, campus (query, all optional)
POST / PATCH / DELETE /poi/ , /poi/{id}	POICreate / POIUpdate (body)
GET /categoria/	–
GET orari/poi/{id_poi}	path param
POST /agenda/	AgendaUtenteCreate (body)
GET /agenda/utente/{id_utente}	solo_futuri (query, opt.)
GET /agenda/imminenti	id_utente, lat, lon (query)
POST /eventi/	EventoCreate (body, incl. lat/lon)
GET /eventi/utente/{id_utente}	path param
GET /eventi/getAll	-
PATCH /eventi/{id}	EventoUpdate (feedback)
GET /recommendations/top_20_ranking	id_utente, lat, lon (query)
POST /recommendations/startup-suggestion	id_utente, lat, lon (query)
GET /geofence/config/{id_utente}	path param
POST /geofence/trigger	GeofenceTriggerRequest (body)
GET /analytics/dashboard	–
GET /analytics/heatmap/pois	id_utente (query, opt.)

2) Authentication
Access control is based on a direct authentication scheme: each user is identified by a unique tuple (name, surname, password) (database constraint). Registration rejects an existing triplet, while login performs an exact match of the three fields and returns the user profile that the client then stores (the user's data) as the session state. As it is not a requirement of the project, the login system has been made as simple as possible.

3) Ranking
The heart of the contextual logic lies in the recommendation service, exposed through the /recommendations router. The app uses two endpoints: GET

/recommendations/top_20_ranking, which returns the 20 points of interest (POIs) with the best score for the user's current location, and POST /recommendations/startup-suggestion, which calculates the ranking, stores the best result as a suggestion event (which will then be notified) and records the motivation along with the score obtained. The calculation of the ranking takes place in three phases:

(a) a spatial scan retrieves all POIs within a radius of 2 km from the user's coordinates, delegating the distance calculation to PostGIS
(b) the user's context vector is constructed by combining explicit category preferences with mobility habits (car/motorcycle -> gas station, bus -> bus stop, train -> station, bike sharing -> bike rental).
(c) each candidate POI *p* at distance *d(p)* (in meters) is assigned a score calculated as *S(p) = B(p) * M(p) * O(p),* where the base score *B(p) = 1000/d(p)* is inversely proportional to the metric distance (limited to 1000 for d <= 0, i.e. when the user is on the geometry of the POI). The preference multiplier *M(p)* is 2.5 if the POI category belongs to the user's context vector and 1 otherwise. The availability factor O(p) is 1 if the POI is currently open and 0 otherwise, effectively excluding closed services. Candidates are then sorted by descending score. Since both the time factor O(p) and the personal factor M(p) directly change the result, the same position produces different rankings at different times of the day and for users with different profiles. As a security measure, if the scoring phase does not produce candidates, the algorithm falls back on returning the nearest open point of interest (POI), if any.
(d) Variable Legend and Final Equation
p = the single POI
S(p) = the total score assigned to the POI
d(p) = the metric distance (in meters) between the user and the POI
cat(p) = the category to which the POI belongs
Cu = the user's context vector
A(p) = the availability status of the POI at the time of the request (open closed).

The Scoring Equation
The algorithm can be expressed in a single multiplicative equation that calculates the score S(p) by combining the three factors (Base, Multiplier, Availability) through a system of conditions: S(p) = B(p) * M(p) * O(p)

Where:
Base Factor B(p): 1000 if d(p) <= 0, otherwise 1000 / d(p)

Multiplier
M(p): 2.5 if cat(p) belongs to Cu, otherwise it is equal to 1

Availability Factor
O(p): 1 if A(p) is open, otherwise it is 0

Fallback condition:

If for all evaluated candidates the score is S(p) = 0 (i.e. there are no open POIs or the algorithm does not produce valid results), the final selection function ignores the score S(p) and simply returns the open POI with the minimum value d(p).

4) Geofencing Services

The /geofence route implements the server-side of the geofencing mechanism. GET /geofence/config/{id_utente} returns the custom set of areas that the mobile client needs to monitor: a circular geofence with a radius of 10m centered on each POI belonging to the user's preferred categories and campus, with a maximum limit of 99 entries to comply with the Android Geofencing API limit. The configuration is then user-specific based on preference. When the client detects a boundary crossing, it invokes the /geofence/trigger POST request, and the service stores an event (geofencing_enter or geofencing_exit) with the trigger location, a human-readable message, and the generation rule in the reason field. This event is returned to the client that displays the message as a local notification.

III.C App Mobile

The mobile application is developed natively for Android in Kotlin, with the user interface based on the Jetpack Compose toolkit and an overall structure that follows the MVVM pattern.

1) UI Structure and Navigation

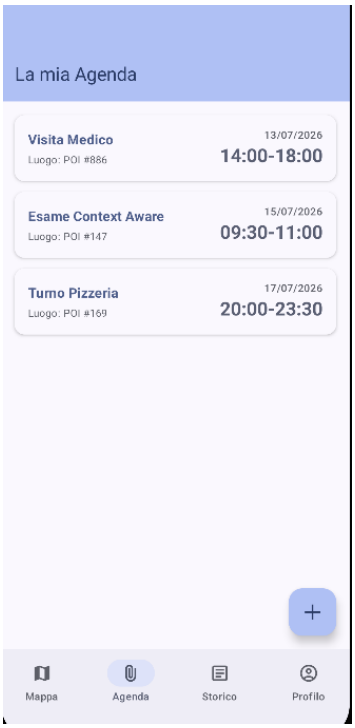
The application flow is orchestrated by a navigation module that provides a quick switch between the five main screens, shown in Figure 3. Each screen associates a Compose View with its dedicated ViewModel:

- Login and Registration (Auth)
  A system entry point that collects credentials, validates them against the backend authentication APIs, and if successful, saves the user profile locally via the SessionManager helper so that user data can be accessed at any time.
- Contextual Map (Mappa)
  The main screen for spatial exploration. View the cartography via Osmdroid's map view, show the markers of points of interest (POIs) and expose the combined filters (category, campus, time range, maximum distance). Proactive suggestions returned by the recommendation engine appear as purgeable cards. To avoid redundant notifications, a new start suggestion (hint card, similar to a notification from below that always appears at least when the application starts) is requested only after the user has moved more than 100 m from the position of the previous one. Tapping on a marker

records a poi_selezionato event.

(Fig. 3 Contextual map screen with a suggestion card)

- My Agenda (Agenda)
  Allows you to view, add, and remove to-do lists, each associated with a point of interest (POI) and time range. Unlike POIs, there are no filters, but only future events are shown.



(Fig. 4 ersonal agenda screen)

- User Profile (Profilo)
  Manages explicit preferences (campus, interest categories, and transportation) that influence user suggestions and geofences.
- Event History (Storico)
  Provides the chronological log of all recorded events (hints, agenda alerts, geofence entry/exit activations), showing for each one the message and the reason that generated it. From this screen, the user can rate each suggestion as Useful or Not Useful (PATCH /eventi/{id}).

(Fig. 5 History screen showing the reason for each event)

For demonstration purposes, the user's location can be simulated by injecting GPS fix into the Android emulator (adb emu geo fix <lon> <lat>), allowing the location to be changed in real-time to show how the system reacts to different contexts.

2) Data Tier

Data access is centralized in domain-specific Repositories, one for each backend entity (e.g. POIs, categories, schedules, agenda, events, preferences). Remote calls are made through Retrofit interfaces configured by the ApiClient helper and in the event of a network failure (java.io.IOException) requests are handled by the local cache. The cache is an SQLite database managed through the Room library, and the necessary tables (points of interest, categories, opening hours, user data, and agenda items) are mapped as dedicated Entities and accessible through their DAOs. These entities are updated whenever the corresponding remote calls are successful.

3) Offline support

Offline mode currently covers map display of cached points of interest, filtering them by campus and category (time-based filtering requires connectivity) and read-only access to the agenda. A separate in-memory singleton (LocationRepository) contains the last validated GPS position and reactively exposes it to ViewModels.

4) Other useful modules

Proactive application behavior relies on three background components.

- The TrackingService, a foreground service that continuously monitors the user's location. To balance responsiveness and battery consumption, updates follow an optimized sampling policy that combines a nominal interval of 15 seconds with a minimum displacement filter of 10m.
- The AgendaNotificationWorker, powered by the WorkManager API, runs periodically every 15 minutes and queries GET /agenda/imminenti to find activities starting in the next 15 minutes, generating a notification and logging an event Avviso_agenda. The endpoint receives, in addition to the user ID, the current coordinates: for each upcoming engagement, the service calculates the distance between the user's location and the associated POI via

PostGIS, including it in the response payload. The notification is therefore contextualized both on the temporal and spatial dimensions.

- The GeofenceManager retrieves the custom geofence configuration when the service starts and registers the corresponding circular regions with the Android Geofencing API (entry and exit transitions). When a boundary crossing is detected, the GeofenceReceiver invokes the /geofence/trigger POST request and displays the contextual message returned as a local notification. A GeoJsonHelper utility handles the analysis of the GeoJSON geometries returned from the backend, including the centroid extraction needed to center the geofences.

III.D Frontend

(Fig. 6 Administrative web dashboard)

To complement the platform, a web dashboard provides the administrative interface for the Points of Interest (POI) catalog and usage analytics visualization (Figure 6). The dashboard interacts asynchronously with the backend REST APIs and performs two macro-functionalities:

1) Management of the POI catalog
The visual core of the dashboard is an interactive map Leaflet.js where the POIs are represented as markers colored by category (with corresponding legend) and organized in layers by category. The same catalogue is also presented in tabular format. A filtering panel allows the administrator to query the catalog by category, campus, and opening time frame, including a shortcut that sets the current time to immediately view the services open at that time. CRUD operations are

accessible through a dedicated module: the administrator can create, modify and delete POIs, specifying their geometry in WKT format (with support for POINT, POLYGON and MULTIPOLYGON shapes automatically normalized to SRID 4326) and managing the weekly opening hours of each POI through the /orari endpoints. Because the dashboard is designed as an internal administration tool, no level of authentication has been implemented.

2) Spatial heatmap
The front-end integrates the leaflet-heat plugin and queries the GET /analytics/heatmap/pois endpoint, which returns the centroids of the POIs involved in the suggestions generated by the system. Each suggestion contributes a point to the heat layer, so that the color gradient (toggleable) reflects the spatial focus of the recommendation activity (events marked with type = "Suggerimento").

3) Usage Analysis
The dashboard also serves as an empirical evaluation tool for the recommendation engine. Four aggregate indicators are shown:

- the number of recorded POIs (a horizontal bar graph)
- the total number of suggestions generated (a bar chart)
- the share of positive and negative feedback (a donut chart)
- the hourly distribution of contextual events (a 24-bucket stacked bar chart, broken down by event type and filterable by date range)

The textual motivation for each suggestion can be consulted on the History screen of the mobile app (Fig. 5), a choice considered more consistent with the recipient of the information.

III.E Containerization

The entire server-side platform is containerized with Docker and orchestrated through Docker Compose. The configuration defines three services that are attached to an isolated internal network:

- The spatial database, based on the official PostGIS/PostGIS image, with its data directory mapped to a volume named for persistence
- the FastAPI backend, built with python:3.12-slim with fixed version dependencies and launched by Uvicorn on port 8000
- the Nginx web server hosting the static dashboard on port 5500, declared as backend dependent.

The entire platform is deployed with a single command (docker compose up --build).

IV. RESULTS

IV.A Demonstration Setup

The functional validation was conducted in an emulated environment on Android Studio, simulating the user's position by injecting GPS fix (adb emu geo fix <lon> <lat>). The scenarios reported mainly refer to the coordinates 44.4949 N, 11.3426 E (Bologna). The database is populated with 1089 real points of interest, extracted by

QGIS from the areas of the Bologna, Forlì and Cesena campuses and subjected to manual review, making the scenarios representative of an actual use of the system.

IV.B Context-aware scenarios

The primary objective of validation is to demonstrate that the same position produces different suggestions as other contextual dimensions vary.

1) Personal factor M(p).

With the same user, position and time, only the preference on the "biblioteca" category has been changed. With the preference active, the suggestion generated is the Gian Paolo Dore Engineering Library, with a score of S(p) = 5.74 (Fig. 7). Once the preference is removed, the multiplier M(p) of the library drops from 2.5 to 1 and the winning POI becomes Villa Baruzziana, with S(p) = 3.32 (Fig. 8). In both cases, the notification reports the motivation and score, making the system's decision inspectable.

(Fig. 7 Suggestion with the "biblioteca" category among the user preferences S = 5.74)

(Fig. 8 Suggestion for the same position without the "biblioteca" category S = 3.32)

2) Time factor

A second test, carried out in the same position on Sunday, shows that the School of Engineering, closed according to ORARIO_POI records, obtains a score of 0 and is therefore "excluded" from the ranking.

IV.C Usage Statistics

The dashboard (Fig. 6) aggregates the recorded events (98 suggestions generated at the time of writing) with distribution by POI and category and a heat map. The dataset of events and feedback has been populated for demonstration purposes, to validate the analytics pipeline (collection, aggregation, visualization) and not to measure the quality of the engine: feedback percentages have no statistical value, also due to the policy for which an ignored suggestion is classified by default as "Not Useful".

IV.D Performance

In cold start, the application completes the download of the POI catalog and the generation of the first suggestion in about 3 seconds. The value is a qualitative observation on an emulator (which

introduces its own overhead: the propagation of a GPS fix takes 3-4 seconds) and not a benchmark; however, confirms that the delegation of spatial calculations to PostGIS (Section II.C) keeps the ranking on ~1100 POIs without perceptible latencies.

IV.E Limitations

- Geofencing coverage.
  To comply with the limit of the Android Geofencing API, the backend returns at most 99 areas, selected from the POIs of the preferred categories located on the user's campus. Entering a POI excluded from the monitored set therefore does not generate any trigger, with coverage being incomplete in the tests. The 10 m range, close to the typical accuracy of GPS, can also delay crossing detection.
- Extension of validation.
  The tests were conducted by a single user in an emulated environment: the validation is functional and does not constitute an experimental evaluation with real users.
- Offline mode.
  Offline support is limited to reading the map only (with the option of filters except for the distance filter and time with POI) and the agenda. The ranking of POIs according to the ranking rule is also not functional offline.

IV.F Future developments

The limitations identified outline the guidelines for the evolution of the platform. First, the explicit feedback collected in the EVENTO table could feed an adaptive ranking mechanism, modulating the multiplier M(p) per category according to the user's historical evaluations, up to the integration of simple supervised learning models trained on the collected feedback, facilitated by the Python ecosystem of the backend. The configuration of geofences could be made dynamic, periodically reselecting the monitored areas based on the user's current position and their opening status (it would be sufficient to reuse the proximity and time verification services already exposed by POIService) so as to mitigate the limit of 99 items and reduce notifications relating to closed services. The integration of location privacy protection techniques (e.g. perturbation of coordinates sent to the backend), strengthening authentication through password hashing and session tokens and, finally, an evaluation campaign with real users to measure the perceived effectiveness of the suggestions are also envisaged.

REFERENCES

[1] S. Ramírez, "FastAPI Documentation.[Online]. Available: https://fastapi.tiangolo.com/learn/

[3] Google, "Jetpack Compose Documentation." [Online]. Available: https://developer.android.com/compose